Project Week 07-08

March 3, 2025

0.0.1 2 Week 7 [Feb 10 - Feb 16]: Collaborative Filtering Recommender System

1. Based on the frequency of the most rated items computed in Week 6, implement the TopPop recommender system, which always recommends the same top-k items sorted decreasingly by the number of "high" ratings (e.g., 3) in the training split, train.tsv.

[50]:
```
import pandas as pd

class TopPop:
    def __init__(self, k=10):
        self.k = k
        self.top_items = None

    def fit(self, file_path):
        # Read the training data (no header assumed)
        df = pd.read_csv(file_path, sep='\t')
        # Keep only the high ratings (e.g., >= 3)
        df_high = df[df['rating'] >= 3]
        # Compute the frequency of high ratings per item and sort in decreasing␣
 ↪order
        counts = df_high['item_id'].value_counts()
        self.top_items = list(counts.index)

    def recommend(self, user_id):
        # Return the top-k items (this model is user-agnostic)
        return self.top_items[:self.k]

top_pop = TopPop(k=10)
top_pop.fit('Data/clean_train.tsv')
```

2. Choose at least one neighborhood-based model and one latent factor model that uses the observed user-item ratings in the training set to predict the unobserved ratings. Report your choice of models.

[51]:
```
from surprise import Dataset, Reader, KNNWithMeans, SVD
import pandas as pd

# Load your training data from the file (in 'Data/clean_train.tsv')
df = pd.read_csv('Data/clean_train.tsv', sep='\t')
```

```
reader = Reader(rating_scale=(1, 5))

# Prepare the dataset using the surprise framework
data = Dataset.load_from_df(df[['user_id', 'item_id', 'rating']], reader)
trainset = data.build_full_trainset()
```

3. Use 5-fold cross-validation on the training set to tune the hyperparameters of the chosen models (similarity measure and number of neighbors for the neighborhood-based model; number of latent factors and number of epochs for the latent factor model).

[52]:
```
from surprise.model_selection import GridSearchCV
import random
import numpy as np

# Set the random seed for reproducibility
random.seed(42)
np.random.seed(42)

# Define parameter grid for KNNWithMeans (neighborhood-based model)
param_grid_knn = {
    'k': [5, 10, 20, 30, 40, 50, 70, 100],
    'verbose': [False],
    'sim_options': {
        'name': ['cosine', 'msd'],
        'user_based': [True]  # We use user-based collaborative filtering
    },
    'random_state': [42]
}

# Define parameter grid for SVD (latent factor model)
param_grid_svd = {
    'n_factors': [3, 5, 10, 20, 50, 100],
    'n_epochs': [5, 10, 20, 50, 100],
    'random_state': [42]
}

# Perform 5-fold cross-validation for KNNWithMeans
grid_knn = GridSearchCV(KNNWithMeans, param_grid_knn, measures=['rmse', 'mae'],␣
 ↪cv=5)
grid_knn.fit(data)

# Perform 5-fold cross-validation for SVD
grid_svd = GridSearchCV(SVD, param_grid_svd, measures=['rmse', 'mae'], cv=5)
grid_svd.fit(data)
```

4. Choose an evaluation measure that is suitable for this task and justify your motivation in using it. Report the optimal hyperparameters together with the scores of your chosen measure, averaged over the 5 folds.

[53]:
```
# We choose RMSE (Root Mean Squared Error) as the evaluation measure

print("\nKNNWithMeans Best Hyperparameters (based on RMSE):")
print(grid_knn.best_params['rmse'])
print("Average RMSE for KNNWithMeans:", grid_knn.best_score['rmse'])

print("\nSVD Best Hyperparameters (based on RMSE):")
print(grid_svd.best_params['rmse'])
print("Average RMSE for SVD:", grid_svd.best_score['rmse'])
```

```
KNNWithMeans Best Hyperparameters (based on RMSE):
{'k': 70, 'verbose': False, 'sim_options': {'name': 'cosine', 'user_based':
True}, 'random_state': 42}
Average RMSE for KNNWithMeans: 0.9100087207501553

SVD Best Hyperparameters (based on RMSE):
{'n_factors': 5, 'n_epochs': 20, 'random_state': 42}
Average RMSE for SVD: 0.8440025192355
```

5. Run the models with the optimal hyperparameters to the whole training set.

[54]:
```
# Re-train the models on the full training set using the optimal␣
 ↪hyperparameters from the grid searches

# For KNNWithMeans
knn_best = grid_knn.best_estimator['rmse']
knn_best.fit(trainset)

# For SVD
svd_best = grid_svd.best_estimator['rmse']
svd_best.fit(trainset)

print("KNNWithMeans model trained with optimal hyperparameters.")
print("SVD model trained with optimal hyperparameters.")
```

```
KNNWithMeans model trained with optimal hyperparameters.
SVD model trained with optimal hyperparameters.
```

6. Use the final models to rank the non-rated items for each user. This ranking will be used for the evaluation part next week.

[55]:
```
from collections import defaultdict

# Build the anti-testset (all user-item pairs not in the training set)
anti_testset = trainset.build_anti_testset()

# Get predictions on the anti-testset using the KNNWithMeans model
predictions = knn_best.test(anti_testset)
```

```python
# Group predictions by user
user_recs = defaultdict(list)
for pred in predictions:
    user_recs[pred.uid].append((pred.iid, pred.est))

# For each user, sort the recommendations in descending order of predicted␣
 ↪rating
for user in user_recs:
    user_recs[user] = [iid for iid, _ in sorted(user_recs[user], key=lambda x:␣
 ↪x[1], reverse=True)]

# Display sample recommendations for a few users
print("Sample recommendations using KNNWithMeans:")
for user, recs in list(user_recs.items())[:5]:
    print(f"User {user}: {recs[:10]}")

# Get predictions on the anti-testset using the SVD model
predictions = svd_best.test(anti_testset)

# Group predictions by user
user_recs = defaultdict(list)
for pred in predictions:
    user_recs[pred.uid].append((pred.iid, pred.est))

# For each user, sort the recommendations in descending order of predicted␣
 ↪rating
for user in user_recs:
    user_recs[user] = [iid for iid, _ in sorted(user_recs[user], key=lambda x:␣
 ↪x[1], reverse=True)]

# Display sample recommendations for a few users
print("Sample recommendations using SVD:")
for user, recs in list(user_recs.items())[:5]:
    print(f"User {user}: {recs[:10]}")
```

```
Sample recommendations using KNNWithMeans:
User AGTVZ7ZSDMTEDLMJGZRC7RFJNDWQ: ['B0C67HCGBR', 'B0BPJ4Q6FJ', 'B0B8M5FJB6',
'B00H35YIJE', 'B079TLFL33', 'B000T9L7W2', 'B007MY5BDI', 'B00H4PEMM6',
'B09YDBKT7M', 'B0002D0Q2W']
User AG44UYFDZHI6FRBQSLDWOGIEOY4A: ['B0C67HCGBR', 'B0B8M5FJB6', 'B000T9L7W2',
'B007MY5BDI', 'B09YDBKT7M', 'B079P9LDHN', 'B0BPKTYTPQ', 'B001W99HE8',
'B000J5UEGQ', 'B01DBS2U9G']
User AE7P3HIBI3UDLCJLUPUZQWYVPEWA: ['B005M0MUQK', 'B0BRS6V8G4', 'B078L11275',
'B09R6KV6QX', 'B07YK57N2M', 'B00RX5HQS4', 'B0BQ4HSKC9', 'B086QM1F75',
'B07R3S93K8', 'B09WF82F1V']
User AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: ['B06XB3FQKB', 'B0BPJ4Q6FJ', 'B0B8M5FJB6',
```

```
'B079TLFL33', 'B007MY5BDI', 'B00H4PEMM6', 'B09YDBKT7M', 'B0BXT384GR',
'B079P9LDHN', 'B0BPKH4HB2']
User AESDNHQRRZEWTZF7A7TFFFTSNY5Q: ['B0BGQZNQ53', 'B07L5B64RG', 'B07Y94MSG9',
'B0CB98SMQR', 'B079NS31NK', 'B00CIHB8FY', 'B000SHQ1QC', 'B0BQ4HSKC9',
'B00IZA1GI2', 'B000U0DU34']
Sample recommendations using SVD:
User AGTVZ7ZSDMTEDLMJGZRC7RFJNDWQ: ['B0BPJ4Q6FJ', 'B00H35YIJE', 'B079TLFL33',
'B000T9L7W2', 'B007MY5BDI', 'B09YDBKT7M', 'B079P9LDHN', 'B0BPKH4HB2',
'B07JCW1KGG', 'B001W99HE8']
User AG44UYFDZHI6FRBQSLDWOGIEOY4A: ['B000T9L7W2', 'B007MY5BDI', 'B09YDBKT7M',
'B079P9LDHN', 'B001W99HE8', 'B01DBS2U9G', 'B07CBT4SYD', 'B09M7DPHCS',
'B08DM9NFLH', 'B0BTC9YJ2W']
User AE7P3HIBI3UDLCJLUPUZQWYVPEWA: ['B0BT2ZRCY2', 'B07DK59QNR', 'B000U0DU34',
'B09G5KLKX2', 'B01DE4D4LU', 'B0B8F6LD9F', 'B0BT9R8MMV', 'B07N5MJDBF',
'B07S19XSPV', 'B0CB98SMQR']
User AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: ['B07N5MJDBF', 'B0BT9R8MMV', 'B0BT2W3TTM',
'B07N2HQ1T7', 'B000U0DU34', 'B09M7BF885', 'B09G5KLKX2', 'B0BRS6V8G4',
'B01DE4D4LU', 'B07DK59QNR']
User AESDNHQRRZEWTZF7A7TFFFTSNY5Q: ['B000U0DU34', 'B0BT2ZRCY2', 'B0BT9R8MMV',
'B0B8F6LD9F', 'B01DE4D4LU', 'B09G5KLKX2', 'B07C9YCY5J', 'B0BQ4HSKC9',
'B07N5MJDBF', 'B0CB98SMQR']
```

0.0.2 3 Week 8 [Feb 17 - Feb 23]: Evaluation of Recommender Systems

In this session, we will discuss how to evaluate a recommender system. Specifically, let us evaluate all your recommender models from Week 7 on the preprocessed test data split studied in Week 6. You need to: - Measure the error of the system's predicted likelihood of rating for the items (Root Mean Square Error, RMSE). - Discuss the limitations of this metric.

[56]:
```python
from surprise import accuracy

# Load the preprocessed test data (assuming same structure as train data: user_id, item_id, rating)
test_df = pd.read_csv('Data/clean_test.tsv', sep='\t')

# Convert the test DataFrame to a list of (user, item, rating) tuples
testset = [tuple(x) for x in test_df[['user_id', 'item_id', 'rating']].values]

# Evaluate SVD model on the test set
svd_predictions = svd_best.test(testset)
print("SVD RMSE on test set:")
accuracy.rmse(svd_predictions, verbose=True)

# Evaluate KNNWithMeans model on the test set
knn_predictions = knn_best.test(testset)
print("KNNWithMeans RMSE on test set:")
accuracy.rmse(knn_predictions, verbose=True)
```

```
SVD RMSE on test set:
RMSE: 0.9795
KNNWithMeans RMSE on test set:
RMSE: 1.0619
```

[56]: 1.0618532415444242

Now, we are interested not in whether the system properly predicts the rating of these items, but rather whether the system gives the best recommendations for each user. To evaluate this, generate the top-k (with k = 10) recommendation for each test user. Based on the top-k recommendation list generated for each user, and using the test data split5, compute: - Hit rate, averaged across users. - Precision@k, averaged across users - Mean Average Precision (MAP@k) - Mean Reciprocal Rank (MRR@k) - Coverage

Discuss the advantages and disadvantages of these metrics. Compare the two types of CF recommender systems and the TopPop system that we have defined so far. Which one works best? Why? What are the advantages and limitations of each approach?

[57]:
```
# Prepare ground truth for each test user (using binary labels)
# Relevant (1) if rating >= 4; not relevant (0) otherwise.
ground_truth = defaultdict(set)
for _, row in test_df.iterrows():
    if row['rating'] >= 4:
        ground_truth[row['user_id']].add(row['item_id'])

# Function to generate top-k recommendations using a given model.
# For personalized models we use the anti-testset from the training set.
def get_top_k(model, trainset, k=10):
    # Build the anti-testset (all unseen items for every user in the trainset)
    anti_testset = trainset.build_anti_testset()
    predictions = model.test(anti_testset)

    recommendations = defaultdict(list)
    for pred in predictions:
        recommendations[pred.uid].append((pred.iid, pred.est))

    # For each user, sort predicted items in descending order and select top-k.
    for user in recommendations:
        recommendations[user] = [iid for iid, _ in␣
 ↪sorted(recommendations[user], key=lambda x: x[1], reverse=True)[:k]]
    return recommendations

# For TopPop (non-personalized), the top-k recommendations are the same for␣
 ↪every user.
def get_top_k_for_top_pop(top_pop, test_users, k=10):
    recs = {}
    top_items = top_pop.top_items[:k]
    for user in test_users:
```

```
        recs[user] = top_items
    return recs

# Generate recommendations for each model.
# Retrieve the list of test users from ground_truth.
test_users = list(ground_truth.keys())

svd_recs = get_top_k(svd_best, trainset, k=10)
knn_recs = get_top_k(knn_best, trainset, k=10)
top_pop_recs = get_top_k_for_top_pop(top_pop, test_users, k=10)

# Get the total catalog size (we use all items from the training split)
total_items = trainset.n_items

# Evaluation function for hit rate, precision@k, MAP@k, MRR@k and coverage.
def evaluate(recs, ground_truth, k=10, total_items=total_items):
    hit_rates = []
    precisions = []
    avg_precisions = []
    reciprocal_ranks = []
    rec_items = set()

    for user, true_items in ground_truth.items():
        recommended = recs.get(user, [])
        rec_items.update(recommended)
        # Hit rate: at least one relevant item recommended.
        hit = 1 if set(recommended) & true_items else 0
        hit_rates.append(hit)
        # Precision@k: fraction of recommended items that are relevant.
        precision = len(set(recommended) & true_items) / k
        precisions.append(precision)
        # Average Precision (AP@k)
        ap = 0
        hit_count = 0
        for i, item in enumerate(recommended):
            if item in true_items:
                hit_count += 1
                ap += hit_count / (i + 1)
        if len(true_items) > 0:
            ap /= len(true_items)
        avg_precisions.append(ap)
        # Reciprocal Rank (MRR@k): rank of the first relevant recommendation.
        rr = 0
        for i, item in enumerate(recommended):
            if item in true_items:
                rr = 1 / (i + 1)
                break
```

```
        reciprocal_ranks.append(rr)

    hit_rate = np.mean(hit_rates)
    precision_at_k = np.mean(precisions)
    map_at_k = np.mean(avg_precisions)
    mrr_at_k = np.mean(reciprocal_ranks)
    # Coverage: unique items recommended over catalog total.
    coverage = len(rec_items) / total_items

    return hit_rate, precision_at_k, map_at_k, mrr_at_k, coverage

# Evaluate each recommender.
svd_metrics = evaluate(svd_recs, ground_truth, k=10)
knn_metrics = evaluate(knn_recs, ground_truth, k=10)
top_pop_metrics = evaluate(top_pop_recs, ground_truth, k=10)

print("SVD:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage =", svd_metrics)
print("KNN:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage =", knn_metrics)
print("TopPop: Hit Rate, Precision@10, MAP@10, MRR@10, Coverage =",␣
 ↪top_pop_metrics)
```

```
SVD:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage = (0.0825242718446602,
0.00825242718446602, 0.00610439651075204, 0.02219140083217753,
0.11787819253438114)
KNN:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage = (0.10922330097087378,
0.011650485436893206, 0.00841981197641613, 0.03142144398212359,
0.587426326129666)
TopPop: Hit Rate, Precision@10, MAP@10, MRR@10, Coverage = (0.25728155339805825,
0.032524271844660196, 0.03386359713699836, 0.11868354137771613,
0.019646365422396856)
```